DEVELOP *like a girl*

ADVANCED KIT / LEARN

Getting started

A practical guide to your first session.

01 Install → 02 Plan → 03 Build

01

Install and verify your kit

Install through the Claude desktop interface below, or use the terminal route. Then open your own project in Claude Code and check that your kit commands are available.

Install in Claude desktop

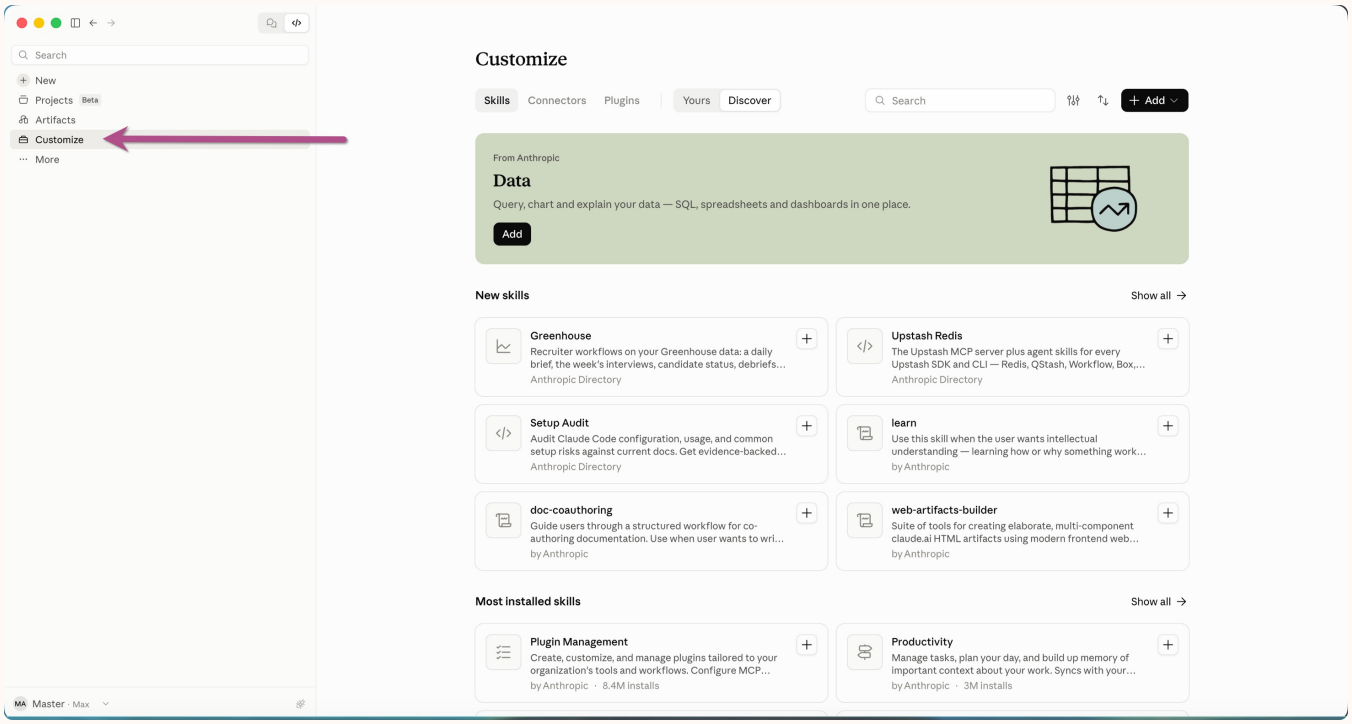
Install and sign in to the Claude desktop app. You do not need a separate CLI installation for this route. The screenshots use **Customize**, which adds plugins to your Claude account. Use the same Claude account when opening Code. See Anthropic's account plugin and sync instructions for account and organization requirements.

The screenshots show the Starter marketplace. For Advanced, enter the **Advanced repository below** and select the `dlag-advanced-` plugins. The navigation is the same; the images are left unchanged.

Private repository: Your GitHub account must have access to Advanced. If Claude asks you to connect GitHub, use the account with access. If the repository does not appear, confirm access before retrying. Advanced does not require Starter.

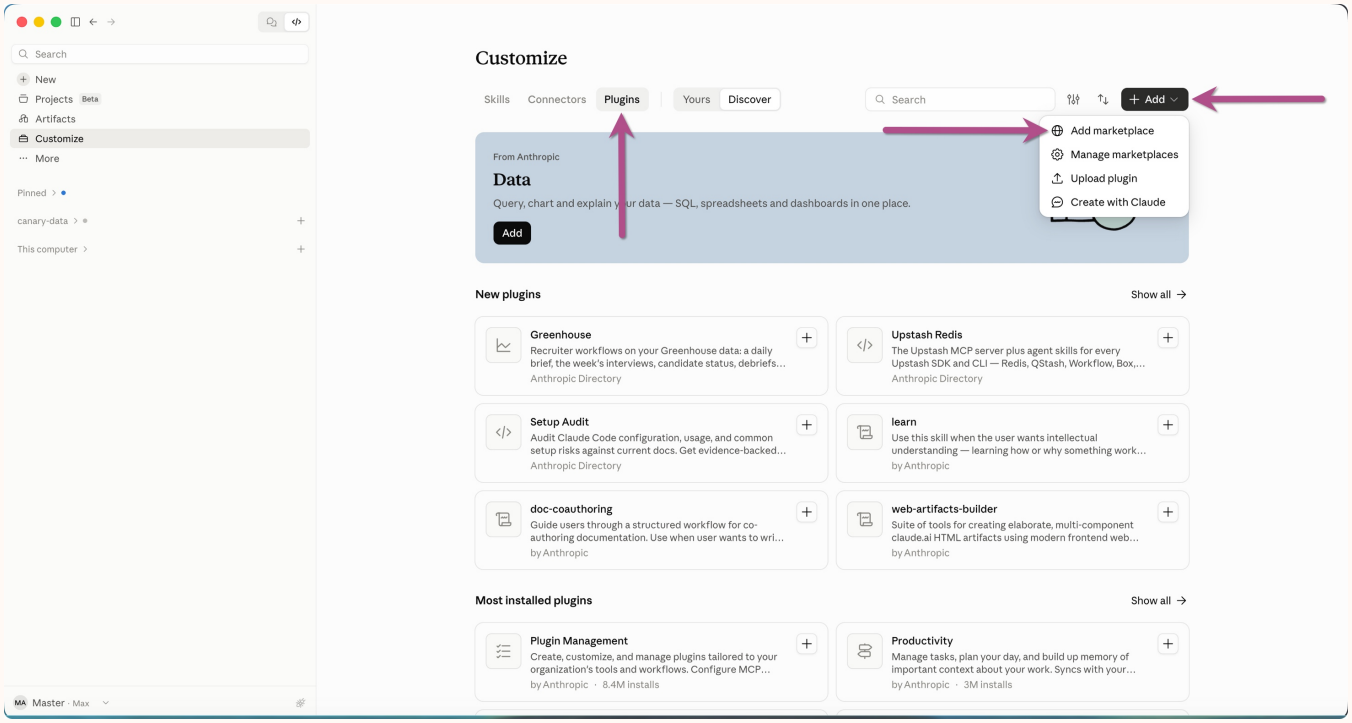
1. Open Customize

In Claude desktop, select **Customize** in the left sidebar.



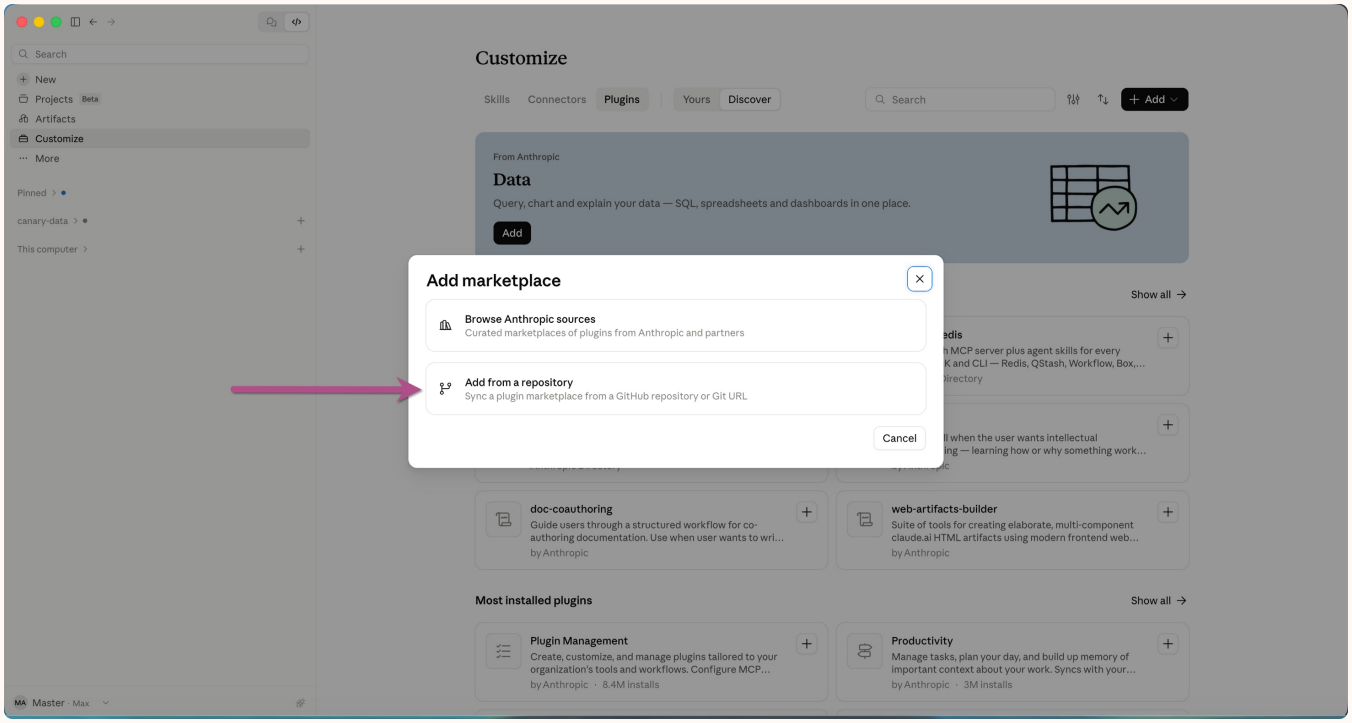
2. Open Plugins and add a marketplace

Select the **Plugins** tab. Open **+ Add** at the upper right, then choose **Add marketplace**.



3. Choose Add from a repository

Select **Add from a repository** in the dialog.

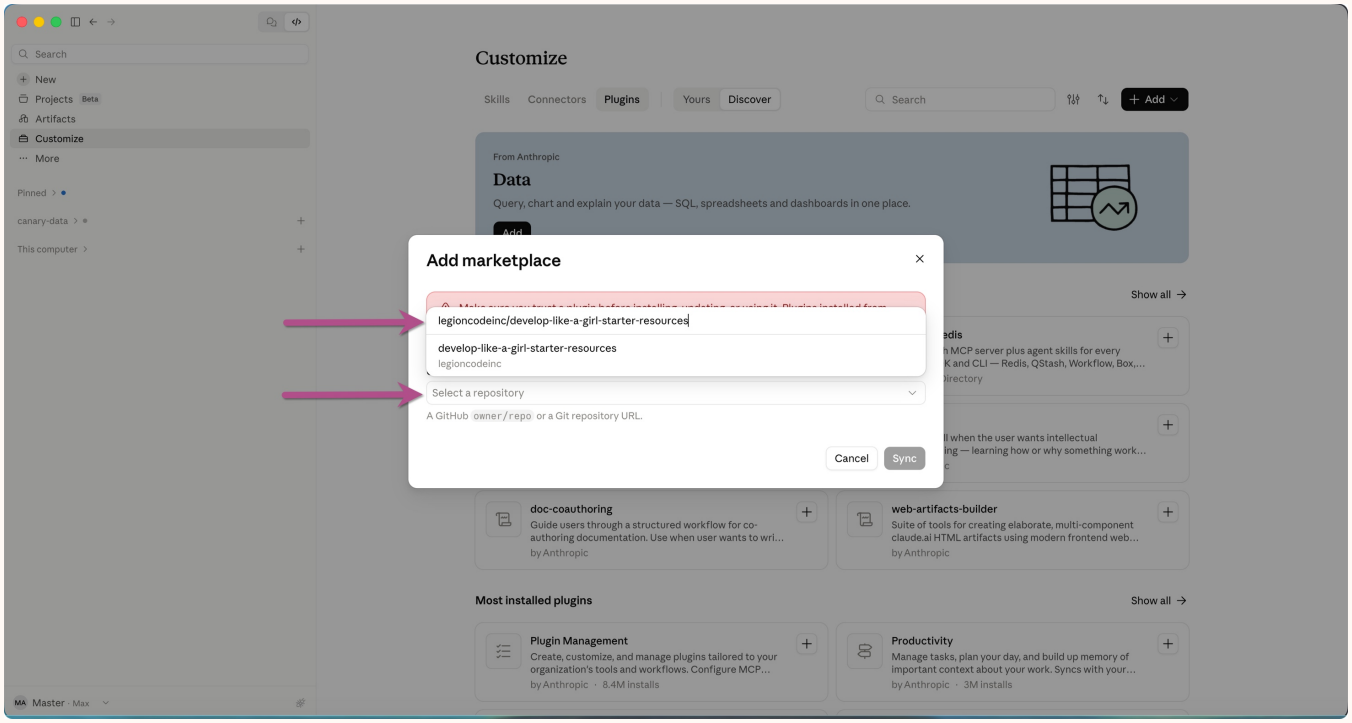


4. Select this kit's repository

Paste this GitHub repository name into the repository field, then select the matching result:

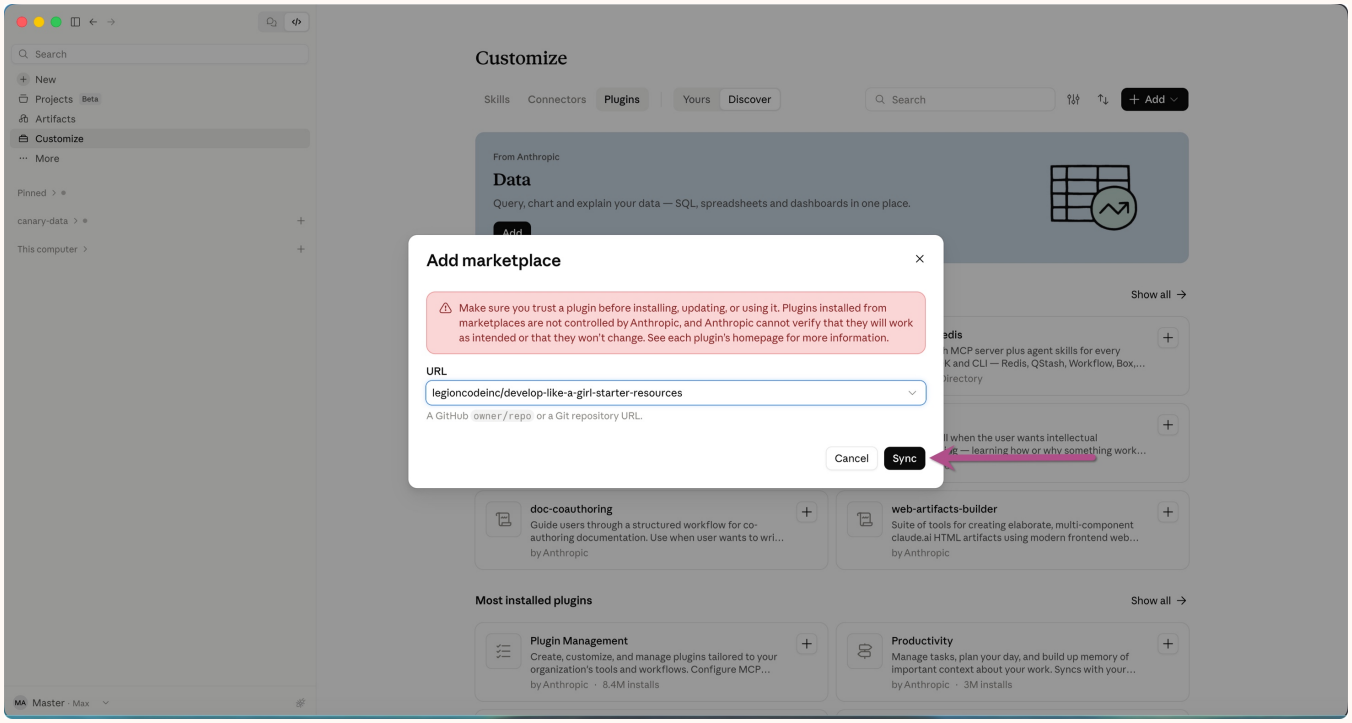
`legioncodeinc/develop-like-a-girl-advanced-resources`

The image below shows the Starter search. Use `develop-like-a-girl-advanced-resources` for this kit.



5. Sync the marketplace

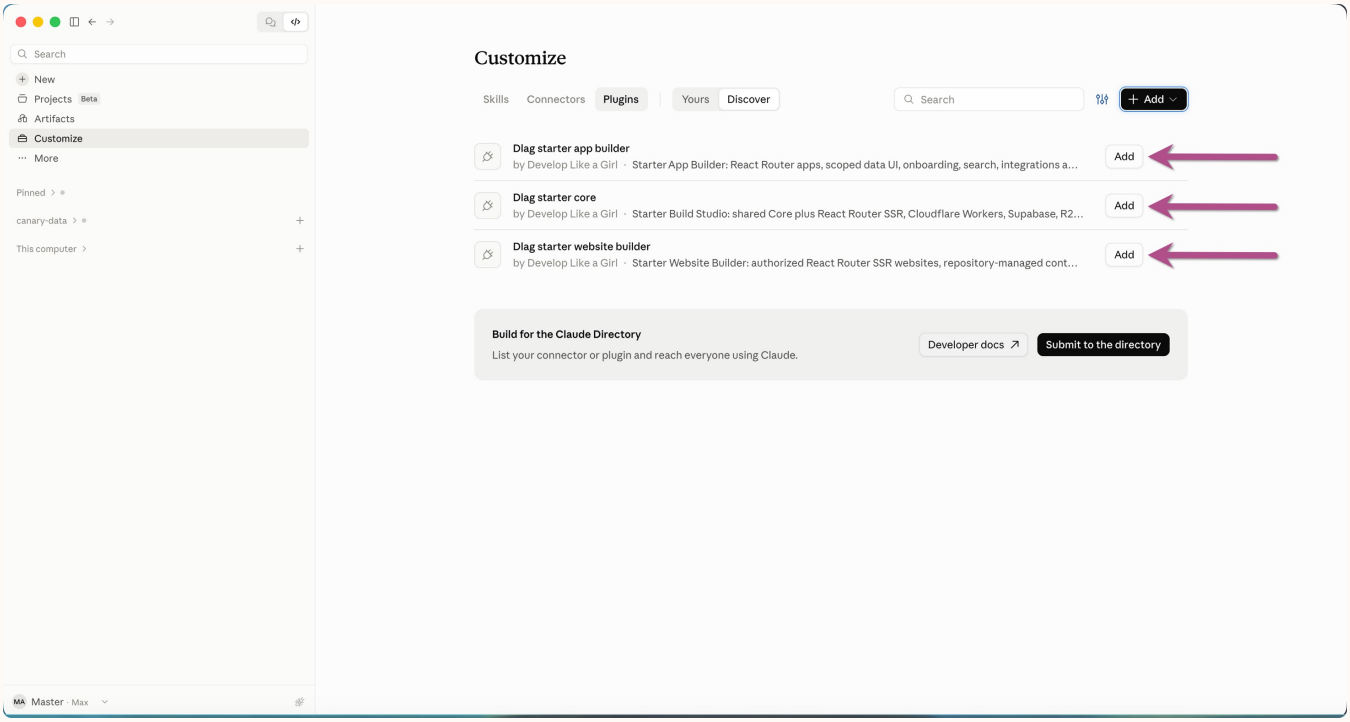
Confirm the selected repository, review the notice, and select **Sync**. This makes the marketplace's plugins available to add.



6. Add Core, then your builder

Add **Dlag advanced core** first. Then add **Dlag advanced website builder**, **Dlag advanced app builder**, or both. Display capitalization may vary; the exact plugin IDs are listed below.

Plugin	Exact ID	Use it for
Core	`dlag-advanced-core`	Shared planning, design, implementation foundations, and reviews.
Website Builder	`dlag-advanced-website-builder`	Public pages, content, and inquiry forms.
App Builder	`dlag-advanced-app-builder`	Signed-in workflows, data screens, search, and integrations.



The screenshot shows the plugins available with **Add** buttons. Select Add for each plugin you want, and confirm it appears under **Yours**. Syncing the marketplace alone does not add all three plugins.

Open your project and verify

Open the **Code** view in Claude desktop, choose a **Local** environment, and select **your own project folder**. Start a new session signed into the same Claude account. In the prompt box, type `/` and look for this kit's commands. For example:

```
/dlag-advanced-core:get-started Inspect this folder and propose a first task.
Keep the proposal in chat.
```

A useful response describes the folder and recommends a next task. This command prepares a setup proposal; application implementation follows separately. The kit commands go in the Code prompt box. Commands

beginning with `claude plugin` belong in a terminal.

Account-installed plugins sync into supported Claude Code versions. If a command is missing, check **Customize → Plugins → Yours**, confirm the same account, and start a fresh Code session. Organization policy or disabled sync can affect availability. See the official sync guidance and desktop command guidance.

You do not need to clone this marketplace or work inside its `plugins/` directory. Your app or website belongs in its own folder.

Install from a terminal

Use this alternative if you prefer the CLI. You do not need to repeat it after a successful desktop account installation and command check.

Have Claude Code CLI and Git available. Follow the CLI setup guide if needed, then check:

```
claude --version
git --version
```

Add the marketplace and install Core:

```
claude plugin marketplace add https://github.com/legioncodeinc/develop-like-a-girl-advanced-resources.git
claude plugin install dlag-advanced-core@dlag-advanced
```

Install the builder you need, or both:

```
claude plugin install dlag-advanced-website-builder@dlag-advanced
claude plugin install dlag-advanced-app-builder@dlag-advanced
```

For this private repository, Git must already be authenticated with an account that has access. These commands install at user scope by default. Local desktop Code sessions on the same computer share those settings. CLI installations remain local to the machine; they do not add plugins to your Claude account. Installation scopes and account sync.

Open a terminal in your project folder and run `claude`, or open that folder in desktop Code. In the terminal CLI session, `/plugin` shows installed plugins. On desktop, type `/` in the Code prompt to check the available kit commands.

If installation stops

Check the exact repository name, account access, and any organization restrictions. An **Add** button means the plugin still needs to be added. If it is added but missing from Code, check account sync and session availability before reinstalling. Do not paste credentials into a prompt or issue. Troubleshooting has the next checks.

Next: Your first session.

Your first session

By the end of this walkthrough, you will have a small project brief and a first task you can approve. Use the example as practice, then replace it with your own idea.

Before you start: Install Core. On desktop, open **Code**, choose **Local**, and select your project folder. In the terminal, open that folder and run `claude`. This exercise can start in an empty folder and does not need a provider account.

1. Describe one person and one job

Send:

```
/dlag-advanced-core:get-started I want a website for a neighborhood pottery class. A
visitor should understand the class and request a place. Inspect this folder, preserve
existing work, and recommend one first task. Keep the proposal in chat.
```

Look for a short account of the folder, any missing information, and a recommendation. It is reasonable for the studio to ask about the class details, audience, or visual direction. Answer the questions that change the first task.

2. Turn the idea into a first version

Send:

```
/dlag-advanced-core:plan-project Plan the first version of the pottery-class website.
Start with one public page showing the class, date, price, and contact details. Use
clearly marked draft copy for missing facts. Save the draft to docs/first-version.md in
this project. If that file already exists, show me how you would update it first. Defer
booking, payment, and email automation.
```

The requested Markdown file should capture the outcome, essential content, deferred ideas, and checks. If the session cannot write files, it should provide the complete draft and say it is unsaved. Confirm the actual saved path before moving on.

3. Make the task testable

Ask:

```
Refine the first task so I can tell when it is finished. Include the mobile layout,
keyboard navigation, clear contact action, and what should happen when content is
missing. Keep the first task limited to the public page.
```

“Build the site” is too broad to review. “A visitor can read the class information on a phone and find the contact action” is something you can check. A plan is still a draft until you accept it.

4. Choose the implementation step

Read the draft, correct the facts, and accept the first task when it matches your intent. Then follow Build a website. That guide uses the Website Builder to author a local slice. Project setup and planning do not scaffold or deploy the application for you.

Before you leave the session

Ask for the saved paths, accepted decisions, open questions, and one next task. Keep this short enough that you can use it to restart tomorrow. If the discussion grows into accounts, checkout, and automation, move those ideas to the deferred list and finish the public page first.

Next: Build a website, or Build an app for a signed-in product.

Continue with the learning hub for website, app, notebook, and release guides. The Markdown guides are the editable source for this edition of the PDF.